Studio Nx

백엔드 211 — Pattern 모델

2026-10-17

Backend(FastAPI/Python) 211 — `Pattern`

항목	내용
파일	`studionx/backend/app/models/pattern.py`
작업	S1-08 아코디언·피아노롤: `patterns` 표 (PRD 02 FR-231, FR-233)
작성	Claude (AI), 2026-10-17, 사용자 검토
이전 · 다음	210 `startSampleEdit` · 212 `MidiNote`

1. 하는 일

MIDI 클립이 가리키는 **노트 묶음(패턴)** 한 건을 담는 표입니다. 노트 자체는 자식 표(`midi_notes`, 212)에 있고, 이 행은 이름과 길이, 스윙만 가집니다.

넣은 것	뜻
`pattern_id`	기본 키 (자동 증가)
`project_id`	어느 프로젝트의 패턴인지 (FK → `projects`, 프로젝트를 지우면 함께 지워짐)
`name`	패턴 이름 (`VARCHAR(100)`)
`color`	클립 색 `#RRGGBB`, 없으면 NULL
`length_tick`	패턴 길이 (1박 = 960 tick, 기본 3840 = 4박)
`swing`	스윙 정도 0.00~1.00 (DB CHECK), 0이면 곧은 박자
`created_at`, `updated_at`	만든 시각·고친 시각

2. 언제 쓰나

쓰는 곳	
`pattern_service.list_patterns` (214)	프로젝트의 패턴 목록을 읽습니다
`pattern_service.get_pattern` (214)	피아노롤이 열 때 이 행과 노트를 함께 읽습니다
`replace_notes` (215)	이 행에 딸린 노트를 통째로 바꿉니다
`update_pattern` (216)	이름·색·길이·스윙을 고칩니다

3. 코드

import (파일 맨 위)

```
from datetime import datetime
from decimal import Decimal
from typing import TYPE_CHECKING

from sqlalchemy import BigInteger, FetchedValue, ForeignKey, Numeric, String, text
from sqlalchemy.dialects.mysql import DATETIME, INTEGER
from sqlalchemy.orm import Mapped, mapped_column, relationship
```

```
from app.db.session import Base

if TYPE_CHECKING:
    from app.models.midi_note import MidiNote
    from app.models.project import Project
```

모델

```
class Pattern(Base):
    __tablename__ = "patterns"

    pattern_id: Mapped[int] = mapped_column(BigInteger, primary_key=True, autoincrement=True)
    project_id: Mapped[int] = mapped_column(
        BigInteger, ForeignKey("projects.project_id", ondelete="CASCADE")
    )
    name: Mapped[str] = mapped_column(String(100))
    color: Mapped[str | None] = mapped_column(String(7))
    length_tick: Mapped[int] = mapped_column(INTEGER(unsigned=True), server_default=text("3840"))
    swing: Mapped[Decimal] = mapped_column(Numeric(3, 2), server_default=text("0.00"))
    created_at: Mapped[datetime] = mapped_column(
        DATETIME(fsp=3), server_default=text("CURRENT_TIMESTAMP(3)")
    )
    updated_at: Mapped[datetime] = mapped_column(
        DATETIME(fsp=3), server_default=text("CURRENT_TIMESTAMP(3)"), server_onupdate=FetchedValue()
    )

    project: Mapped["Project"] = relationship(back_populates="patterns")
    notes: Mapped[list["MidiNote"]] = relationship(
        back_populates="pattern",
        order_by="MidiNote.start_tick, MidiNote.pitch",
        cascade="all, delete-orphan",
        passive_deletes=True,
    )
```

문법 풀이

식	뜻
`INTEGER(unsigned=True)`	MySQL 정수(0 이상)입니다. tick 은 음수가 될 수 없습니다
`Numeric(3, 2)`	소수점 둘째 자리까지(0.00~9.99)입니다. `swing` 0.00~1.00 을 담습니다
`server_default=text("3840")`	값을 안 주면 DB 가 채웁니다. 앱에서 안 넣어도 됩니다
`server_onupdate=FetchedValue()`	DB 의 `ON UPDATE` 가 채우는 칸임을 알려 줍니다
`relationship(..., order_by=...)`	노트를 시작 위치, 음높이 차례로 정렬해 읽습니다
`passive_deletes=True`	삭제는 DB 의 `ON DELETE CASCADE` 에 맡기고 앱이 노트를 하나씩 안 지웁니다
`TYPE_CHECKING`	타입 표시용 import 로 실행 중에는 불러오지 않아 순환 참조를 피합니다

4. 왜 이렇게

선택	이유
노트를 자식 표로	한 패턴에 노트가 많아 패턴 행에 담을 수 없습니다. `midi_notes` 가 `pattern_id` 로 가리킵니다

선택	이유
삭제를 DB 에 맡김	프로젝트를 지우면 패턴, 패턴을 지우면 노트가 `CASCADE` 로 함께 지워집니다. 앱이 빠뜨릴 일이 없습니다
tick 단위	편곡·피아노롤이 1박 = 960 tick 을 씁니다. 화면과 DB 가 같은 단위라 변환이 필요 없습니다

5. 시험

`tests/test_pattern_model.py` 에서 표 이름·칸·기본값·관계가 DB(`assert_matches_db`)와 같은지, 노트가 시작 위치·음높이 차례로 정렬되는지 확인했습니다. 6개가 통과합니다.